

Name: Benedicto, Ruel Jr A.	Date Performed: 2/12/2026
Course/Section: CPE212 - CPE31S1	Date Submitted: 2/12/2026
Instructor: Engr. Robin Valenzuela	Semester and SY: 2nd Sem 2025-2026
Activity 6: Targeting Specific Nodes and Managing Services	
1. Objectives: 1.1 Individualize hosts 1.2 Apply tags in selecting plays to run 1.3 Managing Services from remote servers using playbooks 1.4 Managing Ansible Roles for Modularization	
2. Intended Learning Outcomes: - Differentiate and organize remote hosts into appropriate inventory groups based on their server roles (e.g., web, database, file servers). - Analyze playbook execution results and interpret changes made across different remote servers. - Configure and manage system services remotely using Ansible modules (e.g., service module for starting and enabling services). - Design and apply Ansible Roles to modularize configurations and improve playbook reusability, scalability, and maintainability.	
3. Discussion: In this activity, we try to individualize hosts. For example, we don't want apache on all our servers, or maybe only one of our servers is a web server, or maybe we have different servers like databases or file servers running different things on different categories of servers and that is what we are going to take a look at in this activity. We also try to manage services that do not automatically run using the automations in the playbook. For example, when we install web servers or httpd for CentOS, we notice that the service did not start automatically. Requirement: In this activity, you will need to create another Ubuntu VM and name it Server 3. Likewise, you need to activate the second adapter to a host-only adapter after the installation. Take note of the IP address of Server 3. Make sure to use the command <i>ssh-copy-id</i> to copy the public key to Server 3. Verify if you can successfully SSH to Server 3.	

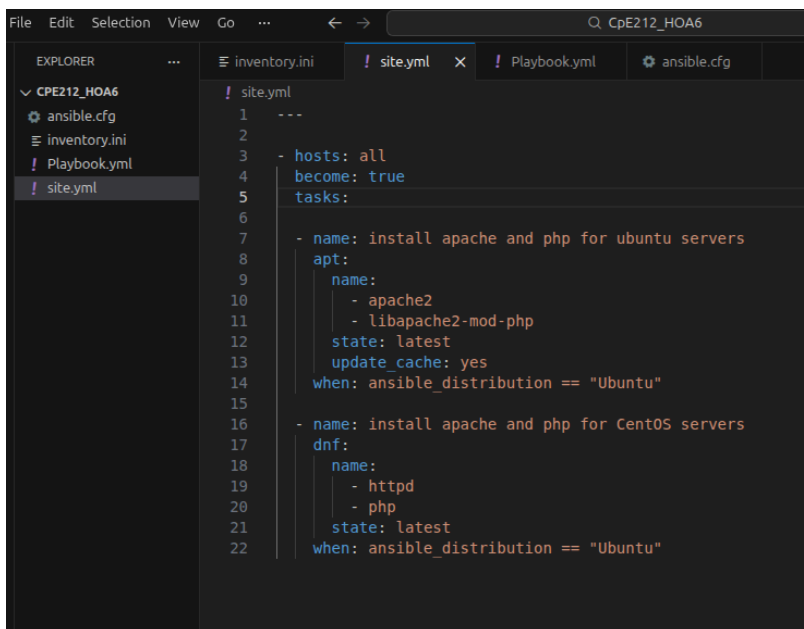
Task 1: Targeting Specific Nodes

1. Create a new playbook and name it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.

```
---
- hosts: all
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```



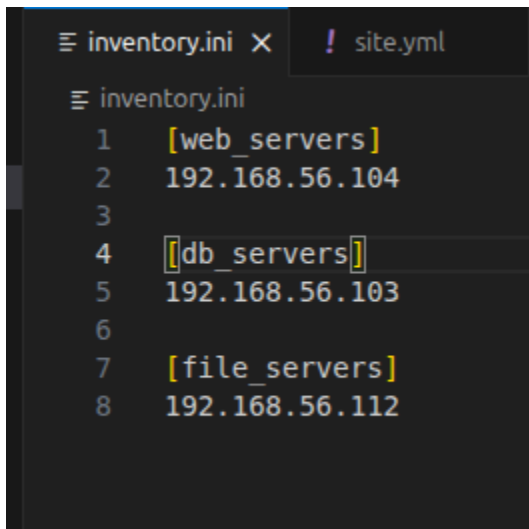
2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```
[web_servers]
192.168.56.120
192.168.56.121

[db_servers]
192.168.56.122

[file_servers]
192.168.56.123
```

Make sure to save the file and exit.



```
inventory.ini x  ! site.yml
inventory.ini
1  [web_servers]
2  192.168.56.104
3
4  [db_servers]
5  192.168.56.103
6
7  [file_servers]
8  192.168.56.112
```

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the *site.yml* by following the image below:

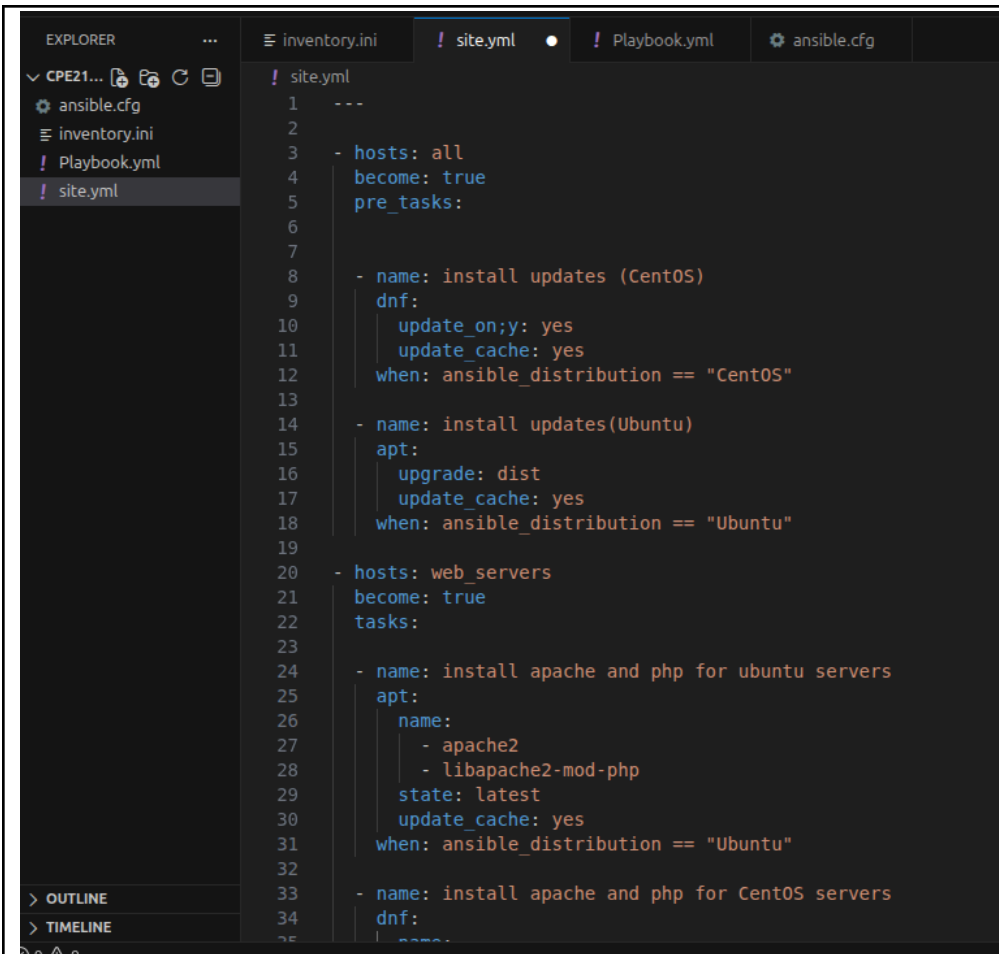
```

- --
- hosts: all
  become: true
  pre_tasks:
    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
        when: ansible_distribution == "CentOS"
    - name: install updates (Ubuntu)
      apt:
        upgrade: dist
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
  tasks:
    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"
    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"

```

Make sure to save the file and exit.



```
1 ---
2
3 - hosts: all
4   become: true
5   pre_tasks:
6
7
8   - name: install updates (CentOS)
9     dnf:
10       update_on;y: yes
11       update_cache: yes
12       when: ansible_distribution == "CentOS"
13
14   - name: install updates(Ubuntu)
15     apt:
16       upgrade: dist
17       update_cache: yes
18       when: ansible_distribution == "Ubuntu"
19
20 - hosts: web_servers
21   become: true
22   tasks:
23
24   - name: install apache and php for ubuntu servers
25     apt:
26       name:
27         - apache2
28         - libapache2-mod-php
29       state: latest
30       update_cache: yes
31       when: ansible_distribution == "Ubuntu"
32
33   - name: install apache and php for CentOS servers
34     dnf:
35       name:
```

The *pre-tasks* command tells the ansible to run it before any other thing. In the *pre-tasks*, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at *web_servers*. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

Run the *site.yml* file and describe the result.

- Let's try to edit again the *site.yml* file. This time, we are going to add plays targeting the other servers. This time we target the *db_servers* by adding it on the current *site.yml*. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```

- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      yum:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true

    - name: install mariadb package (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "Ubuntu"

```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

The screenshot shows an IDE with a file explorer at the top containing 'inventory.ini', 'site.yml', 'site1.yml', 'Playbook.yml', and 'ansible.cfg'. The 'site1.yml' file is open and shows the following content:

```

1 ---
2
3 - hosts: db_servers
4   become: true
5   tasks:
6
7     - name: install mariadb package (CentOS)
8       yum:
9         name: mariadb-server
10        state: latest
11      when: ansible_distribution == "CentOS"
12
13     - name: install mariadb package (Ubuntu)
14       apt:
15         name: mariadb-server
16         state: latest
17      when: ansible_distribution == "Ubuntu"

```

Below the editor, the 'TERMINAL' tab is active, showing the command 'ansible-playbook site1.yml -K' and its output:

```

benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook site1.yml -K

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.103]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.103      : ok=2    changed=0    unreachable=0    failed=0

benedicto@Workstation:~/CpE212_H0A6$

```

- In this part the `site.yml` skipped the CentOS because it doesn't exist and it installs only on the `db_servers`. It's already running because the state is the latest.
5. Go to the remote server (Ubuntu) terminal that belongs to the `db_servers` group and check the status for mariadb installation using the command: `systemctl status mariadb`. Do this on the CentOS server also.

Describe the output.

- In this part of the procedure, in the `site.yml` the hosts specify only this IP address which is the `db_servers` so its the only server that downloads mariadb.

```
benedicto@Server2:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.13 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset:
   Active: active (running) since Thu 2026-02-12 17:46:54 PST; 20min ago
     Docs: man:mariadbd(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 1439 (mariadbd)
    Status: "Taking your SQL requests now..."
     Tasks: 10 (limit: 78187)
    Memory: 109.0M (peak: 112.8M)
       CPU: 1.356s
    CGroup: /system.slice/mariadb.service
            └─1439 /usr/sbin/mariadbd

Feb 12 17:46:54 Server2 mariadbd[1439]: 2026-02-12 17:46:54 0 [Note] InnoDB: L
Feb 12 17:46:54 Server2 mariadbd[1439]: 2026-02-12 17:46:54 0 [Note] InnoDB: B
Feb 12 17:46:54 Server2 mariadbd[1439]: 2026-02-12 17:46:54 0 [Warning] You ne
Feb 12 17:46:54 Server2 mariadbd[1439]: 2026-02-12 17:46:54 0 [Note] Server so
Feb 12 17:46:54 Server2 mariadbd[1439]: 2026-02-12 17:46:54 0 [Note] /usr/sbin
Feb 12 17:46:54 Server2 mariadbd[1439]: Version: '10.11.13-MariaDB-0ubuntu0.24
Feb 12 17:46:54 Server2 systemd[1]: Started mariadb.service - MariaDB 10.11.13
Feb 12 17:46:54 Server2 /etc/mysql/debian-start[1607]: Upgrading MariaDB table
```

6. Edit the `site.yml` again. This time we will append the code to configure installation on the `file_servers` group. We can add the following on our file.

```
- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.

Run the `site.yml` file and describe the result.

```
inventory.ini  ! site.yml  ! site1.yml  ! Playbook.yml  ansible.cfg
! site1.yml
1  ---
2
3  - hosts: file_servers
4    become: true
5    tasks:
6
7      - name: install samba package
8        package:
9          name: samba
10         state: latest

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook site1.yml -K

TASK [Gathering Facts] *****
ok: [192.168.56.112]

TASK [install samba package] *****
changed: [192.168.56.112]

PLAY RECAP *****
192.168.56.112      : ok=2    changed=1    unreachable=0    failed=0
```

```
benedicto@Server3:~$ samba --version
Version 4.19.5-Ubuntu
benedicto@Server3:~$ sudo systemctl status samba
○ samba-ad-dc.service - Samba AD Daemon
   Loaded: loaded (/usr/lib/systemd/system/samba-ad-dc.service; enabled; preset: enabled)
   Active: inactive (dead) (Result: exec-condition) since Thu 2026-02-12 18:14:11 PST; 2min 10s ago
     Condition: start condition unmet at Thu 2026-02-12 18:14:11 PST; 2min 10s ago
       Docs: man:samba(8)
             man:samba(7)
             man:smb.conf(5)
    Process: 23179 ExecCondition=/usr/share/samba/is-configured samba (code=exited, status=1/FAILURE)
      CPU: 17ms

Feb 12 18:14:11 Server3 systemd[1]: Starting samba-ad-dc.service - Samba AD Daemon...
Feb 12 18:14:11 Server3 systemd[1]: samba-ad-dc.service: Skipped due to 'exec-condition'.
Feb 12 18:14:11 Server3 systemd[1]: Condition check resulted in samba-ad-dc.service - Samba AD Daemon being failed.
lines 1-13/13 (END)
[2]+  Stopped                  sudo systemctl status samba
```

The testing of the *file_servers* is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the `site.yml` file. Add tags to the playbook. After the name, we can place the tags: `name_of_tag`. This is an arbitrary command, which means you can use any name for a tag.

```
---  
  
- hosts: all  
  become: true  
  pre_tasks:  
  
    - name: install updates (CentOS)  
      tags: always  
      dnf:  
        update_only: yes  
        update_cache: yes  
        when: ansible_distribution == "CentOS"  
  
    - name: install updates (Ubuntu)  
      tags: always  
      apt:  
        upgrade: dist  
        update_cache: yes  
        when: ansible_distribution == "Ubuntu"
```

```
- hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      tags: apache,apache2,ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      tags: apache,centos,httpd
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      tags: centos, db, mariadb
      dnf:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true

    - name: install mariadb package (Ubuntu)
      tags: db, mariadb, ubuntu
      apt:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "Ubuntu"

- hosts: file_servers
  become: true
  tasks:

    - name: install samba package
      tags: samba
      package:
        name: samba
        state: latest
```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

```
! site.yml x ! main.yml ! site1.yml ! Playbook.yml ansible.cfg
! site.yml
81 - hosts: db_servers
99 - name: "Mariadb- Restarting/Enabling"
100   service:
104
105 - hosts: file_servers
106   become: true
107   tasks:
108
109     - name: install samba package
110       tags: samba
111       package:
112         name: samba
113         state: latest

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook site.yml -K
ok: [192.168.56.103]

TASK [Mariadb- Restarting/Enabling] *****
changed: [192.168.56.103]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.112]

TASK [install samba package] *****
ok: [192.168.56.112]

PLAY RECAP *****
192.168.56.103      : ok=7    changed=1    unreachable=0    failed=0    skipped=3    rescued=0
192.168.56.104      : ok=8    changed=0    unreachable=0    failed=0    skipped=4    rescued=0
192.168.56.112      : ok=6    changed=0    unreachable=0    failed=0    skipped=2    rescued=0

benedicto@Workstation:~/CpE212_H0A6$
```

2. On the local machine, try to issue the following commands and describe each result:

2.1 `ansible-playbook --list-tags site.yml`

```
benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --list-tags site.yml

play #1 (all): all    TAGS: []
TASK TAGS: []

play #2 (web_servers): web_servers    TAGS: []
TASK TAGS: []

play #3 (all): all    TAGS: []
TASK TAGS: [always]

play #4 (web_servers): web_servers    TAGS: []
TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

play #5 (db_servers): db_servers    TAGS: []
TASK TAGS: [centos, db, mariadb, ubuntu]

play #6 (file_servers): file_servers    TAGS: []
TASK TAGS: [samba]
```

2.2 `ansible-playbook --tags centos --ask-become-pass site.yml`

```

benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --tags centos --ask-become-pass site.yml
PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.103]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.112]

PLAY RECAP *****
192.168.56.103      : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    igno
192.168.56.104      : ok=5    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    igno
192.168.56.112      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    igno

benedicto@Workstation:~/CpE212_H0A6$

```

2.3 *ansible-playbook --tags db --ask-become-pass site.yml*

```

benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --tags db --ask-become-pass site.yml
ok: [192.168.56.103]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.103]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.103]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.112]

PLAY RECAP *****
192.168.56.103      : ok=5    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ign
192.168.56.104      : ok=5    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ign
192.168.56.112      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ign

```

2.4 *ansible-playbook --tags apache --ask-become-pass site.yml*

```

benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --tags apache --ask-become-pass site.yml
TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.112]

PLAY RECAP *****
192.168.56.103      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    igno
192.168.56.104      : ok=6    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    igno
192.168.56.112      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    igno

benedicto@Workstation:~/CpE212_H0A6$

```

2.5 *ansible-playbook --tags "apache,db" --ask-become-pass site.yml*

```
benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --tags "apache,db" --ask-become-pass site.yml
PLAY [web_servers] *****
TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install apache2 and php for Ubuntu servers] *****
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]

PLAY [db_servers] *****
TASK [Gathering Facts] *****
ok: [192.168.56.103]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.103]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.103]

PLAY [file_servers] *****
TASK [Gathering Facts] *****
ok: [192.168.56.112]

PLAY RECAP *****
192.168.56.103      : ok=5    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ign
192.168.56.104      : ok=6    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ign
192.168.56.112      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ign
```

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
- name: install apache and php for CentOS servers
  tags: apache,centos,httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

Figure 3.1.1

Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      tags: centos, db,mariadb
      dnf:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true
```

Figure 3.1.2

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command `sudo systemctl stop httpd`. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.
3. Go to the local machine and this time, run the `site.yml` file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.

To automatically enable the service every time we run the playbook, use the command `enabled: true` similar to Figure 7.1.2 and save the playbook.

```

133 tasks:
134
135 - name: install mariadb package (CentOS)
136   tags: centos, db, mariadb
137   dnf:
138     name: mariadb-server
139     state: latest
140   when: ansible_distribution == "CentOS"
141
142 - name: "Mariadb- Restarting/Enabling"
143   service:
144     name: mariadb
145     state: restarted
146
147 ---

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash

```

benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook site.yml -K

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.103]

TASK [Mariadb- Restarting/Enabling] *****
changed: [192.168.56.103]

PLAY RECAP *****
192.168.56.103      : ok=9    changed=2    unreachable=0    failed=0    skipped=4    rescued=0
192.168.56.104      : ok=8    changed=0    unreachable=0    failed=0    skipped=4    rescued=0
192.168.56.112      : ok=6    changed=0    unreachable=0    failed=0    skipped=4    rescued=0

```

○ benedicto@Workstation:~/CpE212_H0A6\$

Supplementary Activity:

1. Creating a Role for Web Servers

```

---
# tasks file for web_servers

- name: install apache and php for ubuntu servers
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

- name: install apache and php for CentOS servers
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: install apache2 and php for Ubuntu servers
  tags: apache, apache2, ubuntu
  apt:
    name:

```

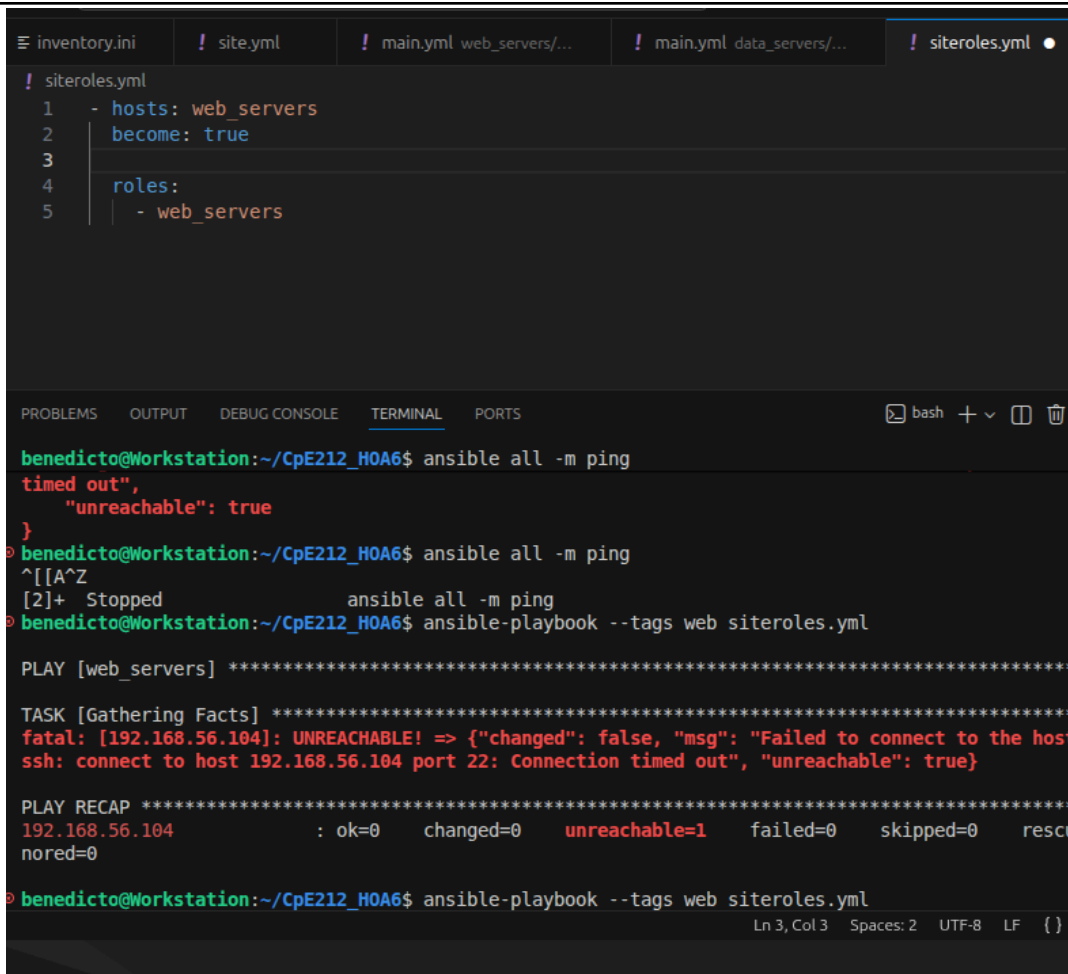
2. Creating a Role for Database Servers

```
---
# tasks file for data_servers
- name: install mariadb package (CentOS)
  tags: centos,db,mariadb
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

- name: install mariadb package (Ubuntu)
  tags: db,mariadb,ubuntu
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: "Mariadb- Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true
```

3. Combining Multiple Roles in a Playbook



```
inventory.ini  ! site.yml  ! main.yml web_servers/...  ! main.yml data_servers/...  ! siteroles.yml ●

! siteroles.yml
1 - hosts: web_servers
2   become: true
3
4   roles:
5     - web_servers

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  bash + v []

benedicto@Workstation:~/CpE212_H0A6$ ansible all -m ping
timed out",
  "unreachable": true
}
benedicto@Workstation:~/CpE212_H0A6$ ansible all -m ping
^[[A^Z
[2]+  Stopped                  ansible all -m ping
benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --tags web siteroles.yml

PLAY [web_servers] *****

TASK [Gathering Facts] *****
fatal: [192.168.56.104]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host
ssh: connect to host 192.168.56.104 port 22: Connection timed out", "unreachable": true}

PLAY RECAP *****
192.168.56.104      : ok=0    changed=0    unreachable=1    failed=0    skipped=0    rescu
nored=0

benedicto@Workstation:~/CpE212_H0A6$ ansible-playbook --tags web siteroles.yml
Ln 3, Col 3  Spaces: 2  UTF-8  LF  {}
```

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?
 - Putting remote servers into groups is important because it helps you organize and manage multiple servers easily. Instead of running commands one-by-one, you can apply the same configuration or task to a whole group (example: web servers, database servers, or backup servers). This saves time, reduces mistakes, and makes automation faster and more efficient.

2. What is the importance of tags in playbooks?
 - **Tags in playbooks are important because they allow you to run specific tasks only, instead of running the entire playbook. For example, if you only want to update packages or restart a service, you can run the playbook with a tag for that task. This makes troubleshooting and updates quicker, safer, and more convenient.**
3. Why do I think some services need to be managed automatically in playbooks?
 - **Some services need to be managed automatically in playbooks because automation ensures the services are always configured correctly, updated, and running properly. It reduces human error, saves time, and makes sure every server has the same setup. This is important especially for critical services like web servers, firewalls, and databases where consistency and reliability are needed.**